

Spring Framework

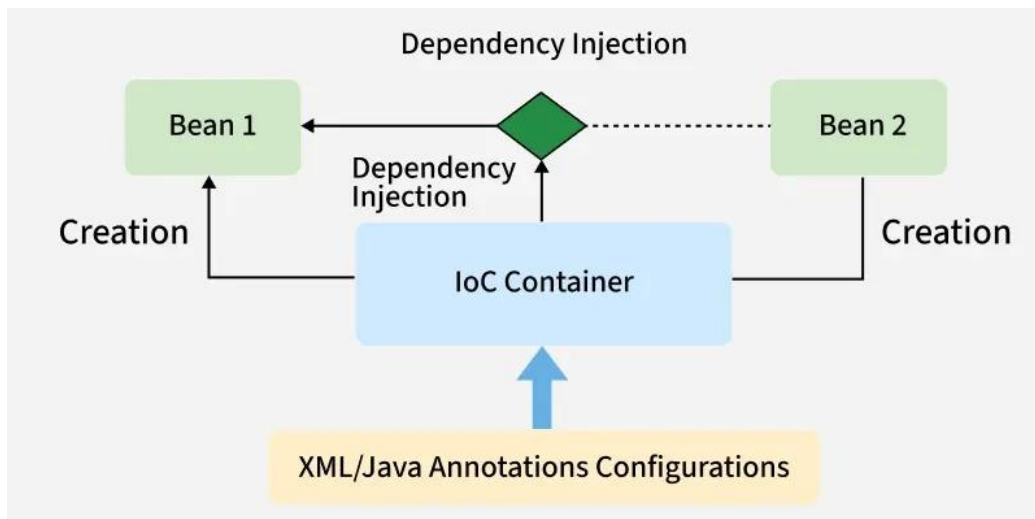
What is Dependency Injection (DI) ?

- Spring DI is a feature that provides objects to classes automatically.
- It removes the need to create objects manually using the new keyword.
- The IoC container manages object creation and dependencies.
- It helps make the application flexible and loosely coupled.

Need for Dependency Injection :-

- Dependency Injection is needed to reduce tight coupling between classes.
- It allows easy management of object creation and dependencies.
- It improves code reusability, flexibility, and testing.
- It makes applications easier to maintain and modify.

Working of Dependency Injection :-



1. Configuration Loading:

The IoC (Inversion of Control) Container reads the configuration details from XML files or Java annotations.

2. Object Creation:

Based on the configuration, the IoC Container creates the required beans (objects) such as Bean 1 and Bean 2.

3. Dependency Injection:

The container injects the dependencies (Bean 2 into Bean 1, for example) automatically, without manual instantiation.

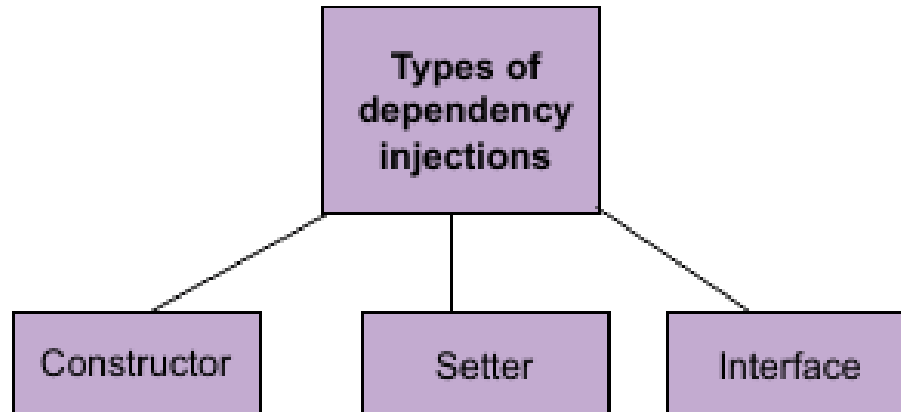
4. Management:

The IoC Container manages the lifecycle and dependencies of all beans in the application.

Types of Dependency Injections :-

There are two primary types of Spring Dependency Injection:

1. Setter Injection
2. Constructor Injection



1. Setter Injection :-

- In this type, dependencies are injected using setter methods.
- It uses the <property> tag in the Spring configuration file.
- It is used when dependencies are optional or can change later.

Benefits of using Setter Injection

1. Flexible to use, it can be changed later.
2. More useful for dependency

2. Constructor Injection :-

- In this type, dependencies are injected through the class constructor.
- It uses the <constructor-arg> tag in the Spring configuration file.
- It is preferred when all dependencies are mandatory at object creation.

Benefits of using Constructor Injection

1. Make your dependencies immutable (final).
2. Make cleaner and testable code.

Difference Between Setter Injections & Constructor Injections :-

Parameter	Setter Injections	Constructor Injections
Type of dependency	Recommended to use when we have some optional dependency. Else our application can provided default business logic code.	Recommended to use when Mandatory/required dependencies are there
Injection method	Through methods	Through Constructors
Creation time	IoC Container first creates the dependent object then the Target Class Object	IoC Container first create the target class object then the dependent object
Efficiency	Slow - because dependencies are injected after object of target class is created	Fast - because Dependencies are injected as soon as the object of target class is created
Recommendation	Not recommended as the first choice	Recommended by spring.io team as your first choice

Implementation of Setter Injection :-

Student.java -

```
package Demo;

public class Student {
    private String name;
    private String address;

    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getAddress() {
        return address;
    }
    public void setAddress(String address) {
        this.address = address;
    }

    public void showdetails()
    {
        System.out.println("Name= "+name);
        System.out.println("Address= "+address);
    }
}
```

Bean Configuration - applicationContext.xml

```
https://www.springframework.org/schema/beans/spring-beans.xsd (xsi:schemaLocation)
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.springframework.org/schema/beans
        https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="s1" class="Demo.Student">
        <property name="name" value="abcd" />
        <property name="address" value="pune" />
    </bean>

    <bean id="s2" class="Demo.Student">
        <property name="name" value="xyz" />
        <property name="address" value="pune" />
    </bean>

</beans>
```

Output :-

```
log4j:WARN No appenders could be found for logger (org.
log4j:WARN Please initialize the log4j system properly.
Name= abcd
Address= pune
```

Implementation of Constructor Injection :-

- 1) using List interface
- 2) using Set interface
- 3) using Map interface

Student.java –

```
package Demo;

import java.util.*;

public class Student {

    private int id;
    private String name;
    private List<String> skills;
    private Set<String> departments;
    private Map<String,String>books;    //key - title, value - author

    Student(){
        System.out.println("Default Constructor");
    }

    Student(int id){
        this.id=id;
    }

    Student(String name){
        this.name=name;
    }

    Student(List<String>skills){
        this.skills=skills;
    }

    Student(Set<String>departments){
        this.departments=departments;
    }

    Student(Map<String,String>books){
        this.books=books;
    }

    Student(int id,String name,List<String> skills,Set<String>departments,Map<String,String>books){
        this.id=id;
        this.name=name;
        this.skills=skills;
        this.departments=departments;
        this.books=books;
    }

    public void viewdetails()
    {
        System.out.println("Id= "+id);
        System.out.println("Name= "+name);
        // System.out.println("Skills= "+skills);
        // System.out.println("Departments= "+departments);
        // System.out.println("Books= "+books);

        Iterator<String> it = skills.iterator();
```

```

        while(it.hasNext())
        {
            System.out.println(it.next());
        }

        Iterator<String> it1 = departments.iterator();
        while(it1.hasNext())
        {
            System.out.println(it1.next());
        }

        Iterator<Map.Entry<String, String>> it2 = books.entrySet().iterator();
        while (it2.hasNext()) {
            Map.Entry<String, String> entry = it2.next();
            System.out.println(entry.getKey() + " : " +entry.getValue());
        }
    }
}

```

Bean Configuration - applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="s1" class="Demo.Student">
        <constructor-arg value="10"></constructor-arg>
        <constructor-arg value="abcd"></constructor-arg>
        <constructor-arg>
            <list>
                <value>HTML</value>
                <value>CSS</value>
                <value>Java</value>
                <value>Spring Framework</value>
            </list>
        </constructor-arg>
        <constructor-arg>
            <set>
                <value>CS</value>
                <value>IT</value>
                <value>ENTC</value>
                <value>MECH</value>
            </set>
        </constructor-arg>
        <constructor-arg>
            <map>
                <entry key="Java" value="Prof. Mehta" />
                <entry key="Python" value="Prof. Shah" />
                <entry key="DBMS" value="Prof. Patil" />
                <entry key="Spring" value="Prof. Rao" />
            </map>
        </constructor-arg>
    </bean>

</beans>

```

Output :-

```
log4j:WARN No appenders could be found for logger i
log4j:WARN Please initialize the log4j system prop
Id= 10
Name= abcd
HTML
CSS
Java
Spring Framework
CS
IT
ENTC
MECH
Java : Prof. Mehta
Python : Prof. Shah
DBMS : Prof. Patil
Spring : Prof. Rao
```